

performed by knowledge compilation into a function $p(q \mid \lambda)$ that computes the probability that q is true according to the logic program with relative frequencies λ . In DeepProbLog, a neural network f_{nn} that outputs a probability distribution $\mathbf{t} = (t_1, \dots, t_n)$ over a set of atoms $\mathbf{a} = (a_1, \dots, a_n)$ is integrated into ProbLog by extending the logic program with $\mathbf{a}$ and the respective probabilities $\mathbf{t}$. The probability of a query q is then given by $p'(q \mid \lambda, f_{nn}(x \mid \mathbf{w}))$, where x is the input of f_{nn} and p' is the function corresponding to the logic program extended with $\mathbf{a}$. Given a set of queries $\mathbf{q}$, input vectors $\mathbf{x}$ and ground-truths $\mathbf{y}$ for all the queries, training is performed by minimizing a loss function that measures the distance between the probabilities predicted by the logic program and the ground-truths, as follows:

$$\mathcal{L}(\mathbf{y}, p'(\mathbf{q} \mid \lambda, f_{nn}(\mathbf{x} \mid \mathbf{w})))$$

The most important difference between DeepProbLog and LTN concerns the logic on which they are based. DeepProbLog adopts probabilistic logic programming. The output of the base neural network is interpreted as the probability of certain atoms being true. LTN instead is based on many-valued logic. The predictions of the base neural network are interpreted as fuzzy truth-values (though previous work [67] also formalizes Real Logic as handling probabilities with relaxed constraints). This difference of logic leads to the second main difference between LTN and DeepProbLog: their inference mechanism. DeepProbLog performs probabilistic inference (based on model counting) while LTN inference consists of computing the truth-value of a formula starting from the truth-values of its atomic components. The two types of inference are incomparable. However, computing the fuzzy truth-value of a formula is more efficient than model counting, resulting in a more scalable inference task that allows LTN to use full first-order logic with function symbols. In DeepProbLog, to perform probabilistic inference, a closed-world assumption is made and a function-free language is used. Typically, DeepProbLog clauses are compiled into Sentential Decision Diagrams (SDDs) to accelerate inference considerably[36], although the compilation step of clauses into the SDD circuit is still costly.

An approach that extends the predictions of a base neural network using abductive reasoning is [14]. Given a neural network $f_{nn}(x \mid \mathbf{w})$ that produces a crisp output $\mathbf{y} \in \{0, 1\}^n$ for n predicates $p_1, \dots, p_n$ and background knowledge in the form of a logic program p , parameters $\mathbf{w}$ of f_{nn} are learned alongside a set of additional rules Δ_C that define a new concept C w.r.t. $p_1, \dots, p_n$ such that, for every object o with features $\mathbf{x}_o$:

$$\begin{aligned} p \cup f_{nn}(\mathbf{x}_o \mid \mathbf{w}) \cup \Delta_C &\models C(o) && \text{if } o \text{ is an instance of } C \\ p \cup f_{nn}(\mathbf{x}_o \mid \mathbf{w}) \cup \Delta_C &\models \neg C(o) && \text{if } o \text{ is not an instance of } C \end{aligned} \quad (62)$$

The task is solved by iterating the following three steps:

1. Given the predictions of the neural network $\{f_{nn}(\mathbf{x}_o \mid \mathbf{w})\}_{o \in O}$ on the set O of training objects, search for the best Δ_C that maximize the number of objects for which (62) holds;
2. For each object o , compute by abduction on $p \cup \Delta_C$, the explanation $\mathbf{p}'(o)$;
3. Retrain f_{nn} with the training set $\{\mathbf{x}_o, \mathbf{p}'(o)\}_{o \in O}$.

Differently from LTN, in [14] the optimization is done separately in an iterative way. The semantics of the logic is crisp, neither fuzzy nor probabilistic, and therefore not fully differentiable. Abductive reasoning is adopted, which is a potentially relevant addition for comparison with symbolic ML and Inductive Logic Programming approaches [50].

Various other loosely-coupled approaches have been proposed recently such as [44], where image classification is carried out by a neural network in combination with reasoning from text data for concept learning at a higher level of abstraction than what is normally possible with pixel data alone. The proliferation of such approaches has prompted Henry Kautz to propose a taxonomy for neurosymbolic AI in [34] (also discussed in [17]), including recent work combining neural networks with graphical models and graph neural networks [4, 40, 58], statistical relational learning [21, 55], and even verification of neural multi-agent systems [2, 8].

6. Conclusions and Future Work

In this paper, we have specified the theory and exemplified the reach of Logic Tensor Networks as a model and system for neurosymbolic AI. LTN is capable of combining approximate reasoning and deep learning, knowledge and data.

For ML practitioners, learning in LTN (see Section 3.2) can be understood as optimizing under first-order logic constraints relaxed into a loss function. For logic practitioners, learning is similar to inductive inference: given a theory, learning makes generalizations from specific observations obtained from data. Compared to other neuro-symbolic architectures (see Section 5), the LTN framework has useful properties for gradient-based optimization (see Section 2.4) and a syntax that supports many traditional ML tasks and their inductive biases (see Section 4), all while remaining computationally efficient (see Table 1).

Section 3.4 discussed reasoning in LTN. Reasoning is normally under-specified within neural networks. Logical reasoning is the task of proving if some knowledge follows from the facts which are currently known. It is traditionally achieved semantically using model theory or syntactically via a proof system. The current LTN framework approaches reasoning semantically, although it should be possible to use LTN and querying alongside a proof system. When reasoning by refutation in LTN, to find out if a statement ϕ is a logical consequence of given data and knowledge-base $\mathcal{K}$, a proof by refutation attempts to find a semantic counterexample where $\neg\phi$ and $\mathcal{K}$ are satisfied. If the search fails then ϕ is assumed to hold. This approach is efficient in LTN when we allow for a direct search to find counterexamples via gradient-descent optimization. It is assumed that ϕ , the statement to prove or disprove, is known. Future work could explore automatically inducing which statement ϕ to consider, possibly using syntactical reasoning in the process.

The paper formalizes Real Logic, the language supporting LTN. The semantics of Real Logic are close to the semantics of Fuzzy FOL with the following major differences: 1) Real Logic domains are typed and restricted to real numbers and real-valued tensors, 2) Real Logic variables are sequences of fixed length, whereas FOL variables are a placeholder for any individual in a domain, 3) Real Logic relations are interpreted as mathematical functions, whereas Fuzzy Logic relations are interpreted as fuzzy set membership functions. Concerning the semantics of connectives and quantifiers, some LTN implementations correspond to semantics for t-norm fuzzy logic, but not all. For example, the conjunction operator in stable product semantics is not a t-norm, as pointed out at the end of Section 2.4.

Integrative neural-symbolic approaches are known for either seeking to bring neurons into a symbolic system (neurons into symbols) [41] or to bring symbols into a neural network (symbols into neurons) [60]. LTN adopts the latter approach but maintaining a close link between the symbols and their grounding into the neural network. The discussion around these two options - neurons into symbols vs. symbols into neurons - is likely to take center stage in the debate around neurosymbolic AI in the next decade. LTN and related approaches are well placed to play an

important role in this debate by offering a rich logical language tightly coupled with an efficient distributed implementation into TensorFlow computational graphs.

The close connection between first-order logic and its implementation in LTN makes LTN very suitable as a model for the neural-symbolic cycle [27, 29], which seeks to translate between neural and symbolic representations. Such translations can take place at the level of the structure of a neural network, given a symbolic language [27], or at the level of the loss functions, as done by LTN and related approaches [13, 45, 46]. LTN opens up a number of promising avenues for further research:

Firstly, a continual learning approach might allow one to start with very little knowledge, build up and validate knowledge over time by querying the LTN network. Translations to and from neural and symbolic representations will enable reasoning also to take place at the symbolic level (e.g. alongside a proof system), as proposed recently in [70] with the goal of improving fairness of the network model.

Secondly, LTN should be compared in large-scale practical use cases with other recent efforts to add structure to neural networks such as the neuro-symbolic concept learner [44] and high-level capsules which were used recently to learn the *part-of* relation [38], similarly to how LTN was used for semantic image interpretation in [19].

Finally, LTN should also be compared with Tensor Product Representations, e.g. [59], which show that state-of-the-art recurrent neural networks may fail at simple question-answering tasks, despite achieving very high accuracy. Efforts in the area of transfer learning, mostly in computer vision, which seek to model systematicity could also be considered a benchmark [5]. Experiments using fewer data and therefore lower energy consumption, out-of-distribution extrapolation, and knowledge-based transfer are all potentially suitable areas of application for LTN as a framework for neurosymbolic AI based on learning from data and compositional knowledge.

Acknowledgement

We would like to thank Benedikt Wagner for his comments and a number of productive discussions on continual learning, knowledge extraction and reasoning in LTNs.

References

- [1] Martín Abadi, Ashish Agarwal, Paul Barham, Eugene Brevdo, Zhifeng Chen, Craig Citro, Greg S. Corrado, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Ian Goodfellow, Andrew Harp, Geoffrey Irving, Michael Isard, Yangqing Jia, Rafal Jozefowicz, Lukasz Kaiser, Manjunath Kudlur, Josh Levenberg, Dandelion Mané, Rajat Monga, Sherry Moore, Derek Murray, Chris Olah, Mike Schuster, Jonathon Shlens, Benoit Steiner, Ilya Sutskever, Kunal Talwar, Paul Tucker, Vincent Vanhoucke, Vijay Vasudevan, Fernanda Viégas, Oriol Vinyals, Pete Warden, Martin Wattenberg, Martin Wicke, Yuan Yu, and Xiaoqiang Zheng. TensorFlow: Large-scale machine learning on heterogeneous systems, 2015. Software available from tensorflow.org.
- [2] Michael Akintunde, Elena Botoeva, Panagiotis Kouvaros, and Alessio Lomuscio. Verifying strategic abilities of neural multi-agent systems. In *Proceedings of 17th International Conference on Principles of Knowledge Representation and Reasoning, KR2020, Rhodes, Greece, September 2020*.